

INF672 · MACHINE LEARNING · GL07

Preprocesamiento de datos

Tratamiento de faltantes · Codificación categórica · Escalado y normalización · Pipelines

Guía de laboratorio · Facultad de Ciencias Puras · Carrera de Ingeniería Informática · UATF

Unidad 2.3 · Preparación de datos para el aprendizaje automático

Campo	Detalle
Asignatura	INF672 — Machine Learning
Guía	GL07 — Preprocesamiento de datos
Unidad temática	2.3 · Preparación de datos (faltantes, codificación, escalado, pipelines)
Dataset	solicitudes_credito.csv (800 solicitudes · 17 columnas)
Herramientas	Python · pandas · NumPy · scikit-learn · Matplotlib · Seaborn
Modalidad	Laboratorio guiado + trabajo autónomo del estudiante
Duración estimada	1 sesión de laboratorio
Entregable	Cuaderno .ipynb ejecutado

Competencia de la unidad

Transformar un conjunto de datos crudo, con valores faltantes, texto inconsistente y escalas dispares, en una matriz numérica limpia y lista para entrenar un modelo, aplicando en cada decisión el criterio técnico correcto y evitando la fuga de información ([data leakage](#)).

1 Objetivos de aprendizaje

Al terminar esta guía, el estudiante será capaz de:

1. **Diagnosticar** valores faltantes e identificar su *mecanismo* (MCAR, MAR, estructural/MNAR) y elegir la estrategia de imputación adecuada a cada caso.
2. **Codificar** variables categóricas nominales y ordinales, distinguiendo cuándo usar `OneHotEncoder`, `OrdinalEncoder` o *frequency encoding*.
3. **Escalar y normalizar** variables numéricas y decidir entre `StandardScaler`, `MinMaxScaler` y `RobustScaler` según la presencia de *outliers*.
4. **Componer** todo lo anterior en un `Pipeline` con `ColumnTransformer`, evitando la fuga de datos y dejando el preprocesamiento reproducible.

CÓMO USAR ESTA GUÍA

Cada sección alterna **teoría** (definiciones y criterios), un **ejercicio** resuelto paso a paso con su código y salida real, y **advertencias** sobre los errores más comunes. Las celdas `In [ ]`: contienen el código a ejecutar; las celdas `Out [ ]`: muestran el resultado esperado sobre el dataset de esta guía.

2 ¿Por qué preprocesar? El 80 % del trabajo

Un algoritmo de aprendizaje automático no «entiende» texto, ni categorías, ni celdas vacías: solo opera sobre **números**. La calidad de esos números determina el techo de lo que cualquier modelo puede lograr. Existe una regla empírica muy citada en la práctica: en un proyecto real de ciencia de datos, cerca del **80 % del esfuerzo** se va en obtener, limpiar y preparar los datos, y solo el 20 % restante en el modelado propiamente dicho. Esta guía trabaja ese 80 %.

El preprocesamiento no es un trámite mecánico: cada transformación es una **decisión con consecuencias**. Rellenar un vacío con la media puede inventar información falsa; codificar mal una categoría puede introducir un orden inexistente; escalar usando todo el dataset puede filtrar el futuro dentro del entrenamiento. Aprender preprocesamiento es, sobre todo, aprender a **elegir bien**.

2.1 • El dataset — `solicitudes_credito.csv`

Trabajaremos con `solicitudes_credito.csv`: 800 solicitudes de crédito de una entidad financiera boliviana. La variable objetivo es `aprobado` (1 = crédito aprobado, 0 = rechazado). El diccionario de variables es el siguiente:

Variable	Tipo	Descripción / dominio
<code>id_solicitud</code>	ID	Identificador único (no se usa para modelar).
<code>edad</code>	Númérica	Edad del solicitante (18–70).
<code>genero</code>	Nominal	Femenino / Masculino.

Variable	Tipo	Descripción / dominio
estado_civil	Nominal	Casado, Soltero, Divorciado, Viudo, Unión Libre (texto sucio y faltantes).
nivel_educacion	Ordinal	Primaria < Secundaria < Técnico < Licenciatura < Posgrado.
ciudad	Nominal	15 ciudades (alta cardinalidad).
tipo_empleo	Nominal	Dependiente, Independiente, Empleado Público, Jubilado, Desempleado.
antiguedad_laboral_meses	Numérica	Meses en el empleo (1–480). Falta en desempleados.
ingreso_mensual_bs	Numérica	Ingreso mensual en Bs. Con outliers y faltantes.
monto_solicitado_bs	Numérica	Monto pedido en Bs (3.044–400.000).
plazo_meses	Numérica	Plazo del crédito en meses (6–60).
num_dependientes	Numérica	Personas a cargo (0–6).
puntaje_crediticio	Numérica	Score crediticio (332–850). Con faltantes.
tiene_vivienda_propia	Nominal	Sí / No (texto sucio: 'sí', 'NO'...).
historial_mora	Nominal	Sí / No.
nivel_riesgo	Ordinal	Bajo < Medio < Alto.
aprobado	Objetivo	0 = rechazado · 1 = aprobado (330 aprobados / 800 = 41,2 %).

DEFINICIÓN · TIPOS DE VARIABLE

Numérica: magnitud continua o de conteo (edad, ingreso). **Categórica nominal:** etiquetas sin orden (ciudad, género). **Categórica ordinal:** etiquetas con un orden natural (nivel educativo, riesgo). El tipo determina la técnica de codificación: confundir nominal con ordinal es uno de los errores más comunes.

3 Preparación del entorno

Importamos la pila científica y cargamos el CSV. La paleta «*cuaderno científico*» (índigo, púrpura, coral) se reutiliza en todos los gráficos para mantener la identidad visual de la materia.

NOTA • REQUISITO

Coloca `solicitudes_credito.csv` en la **misma carpeta** que este cuaderno antes de ejecutar.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option("display.max_columns", 30)
pd.set_option("display.width", 120)
sns.set_theme(style="whitegrid")

# Paleta "cuaderno científico" para los gráficos
INDIGO, PURPURA, CORAL = "#4F46E5", "#7E22CE", "#F43F5E"

df = pd.read_csv("solicitudes_credito.csv")
print("Dimensiones:", df.shape)
df.head()
```

```
Out
[1]: Dimensiones: (800, 17)

(se muestran las 5 primeras filas con df.head())
```

3.1 • Radiografía inicial

Antes de transformar nada, se «radiografía» el dataset con dos comandos que todo cuaderno de ML debería empezar ejecutando: `df.info()` (tipos y no-nulos por columna) y `df.describe()` (resumen estadístico de las numéricas).

```
In [2]: df.info()
```

```
Out
[2]: <class 'pandas.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype
---  -
 3   estado_civil          776 non-null   str
 7   antigüedad_laboral_meses  741 non-null   float64
 8   ingreso_mensual_bs     683 non-null   float64
12   puntaje_crediticio     733 non-null   float64
... (columnas completas al ejecutar)
dtypes: float64(4), int64(5), str(8)
```

Ya en `info()` se detecta el primer problema: cuatro columnas tienen menos de 800 valores no nulos. Hay faltantes.

```
In [3]: # Resumen estadístico de las variables numéricas
df.describe().round(1)
```

El resumen estadístico (redondeado) sobre las variables numéricas:

variable	count	media	std	min	25%	50%	75%	max
edad	800	44.4	15.2	18	31	44	58	70
antigüedad_lab.	741	254.6	137.1	1	138	268	371	480
ingreso_mensual	683	9683.4	18617.5	800	4882	7156	9640	185877
monto_solicitado	800	57746	54732	3044	25477	46436	71923	400000
plazo_meses	800	24.9	13.8	6	12	24	36	60
num_dependientes	800	3.0	2.0	0	1	3	5	6
puntaje_credit.	733	652.8	92.6	332	597	655	716	850

CUIDADO • SEÑAL DE OUTLIERS

En `ingreso_mensual_bs` la media (9.683) es mucho mayor que la mediana (7.156) y el máximo (185.877) está muy lejos del percentil 75 (9.640). Esa asimetría delata **valores extremos**: será decisivo al escalar.

EJERCICIO • 0.1 • PRIMER DIAGNÓSTICO

Antes de tocar nada: ¿cuántos valores faltan por columna (en cantidad y en %)? ¿Qué columnas categóricas tienen valores *inconsistentes* (mayúsculas / espacios)?

Solución

```
In [4]: faltantes = pd.DataFrame({
        "n_faltantes": df.isna().sum(),
        "pct_faltantes": (df.isna().mean()*100).round(1)
    })
    faltantes[faltantes["n_faltantes"] > 0].sort_values("pct_faltantes",
    ascending=False)
```

Solo cuatro columnas tienen huecos. Ordenadas por porcentaje:

columna	n_faltantes	% faltantes
<code>ingreso_mensual_bs</code>	117	14.6 %
<code>puntaje_crediticio</code>	67	8.4 %
<code>antigüedad_laboral_meses</code>	59	7.4 %
<code>estado_civil</code>	24	3.0 %

```
In [5]: # Categóricas sospechosas: mismos valores escritos de formas distintas
print("tiene_vivienda_propia:",
      sorted(df["tiene_vivienda_propia"].dropna().unique()))
print("estado_civil          :", sorted(df["estado_civil"].dropna().unique()))
```

```
Out
[4]: tiene_vivienda_propia: ['NO', 'No', 'Sí', 'si']
     estado_civil          : [' CASADO', ' DIVORCIADO', ' SOLTERO',
                             ' UNIÓN LIBRE', ' VIUDO', 'Casado',
                             'Divorciado', 'Soltero', 'Unión Libre', 'Viudo']
```

CUIDADO

' CASADO', 'Casado' y 'casado' se tratarían como categorías **distintas** al codificar. Hay que limpiar el texto **antes** de codificar (lo haremos en §6).

4 Tratamiento de valores faltantes • Unidad 2.3.1

La pregunta no es «¿con qué relleno el hueco?» sino «*¿por qué* falta este dato?». El motivo de la ausencia —su **mecanismo**— determina la estrategia correcta. Imputarlo todo con la media es el atajo que arruina modelos.

DEFINICIÓN • MECANISMO DE DATOS FALTANTES

MCAR (faltan completamente al azar): la ausencia no depende de nada. **MAR** (faltan al azar condicionado): la ausencia depende de *otra* variable observada. **Estructural / MNAR**: el dato no existe, o su ausencia depende de su propio valor. **La estrategia correcta depende del mecanismo: no todo se imputa con la media.**

¿Por qué importa el mecanismo?

- **MCAR** — Ejemplo: un sensor que falla al azar. Imputar con la mediana global apenas sesga; es el caso más benigno.
- **MAR** — Ejemplo: los independientes declaran menos su ingreso. La ausencia depende de `tipo_empleo`, así que conviene imputar **por grupo** (mediana de cada tipo de empleo), no una mediana global que mezclaría poblaciones distintas.
- **Estructural / MNAR** — Ejemplo: un desempleado no tiene antigüedad laboral. El dato *no existe*; imputar una mediana inventaría meses de trabajo que nunca ocurrieron. Se marca con una **variable indicadora** y se rellena con un valor neutro.

EJERCICIO • 1.1 • VISUALIZAR EL PATRÓN DE FALTANTES

Genera un gráfico de barras con el % de faltantes por columna y un mapa de calor que muestre *dónde* caen los huecos.

Solución

```
In [6]: fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

pct = (df.isna().mean()*100)
pct = pct[pct > 0].sort_values()
axes[0].barh(pct.index, pct.values, color=INDIGO)
axes[0].set_title("% de valores faltantes por columna")
axes[0].set_xlabel("%")

sns.heatmap(df.isna(), cbar=False, cmap=["#E5E7EB", CORAL], ax=axes[1])
axes[1].set_title("Mapa de faltantes (coral = ausente)")
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=90, fontsize=7)
plt.tight_layout(); plt.show()
```

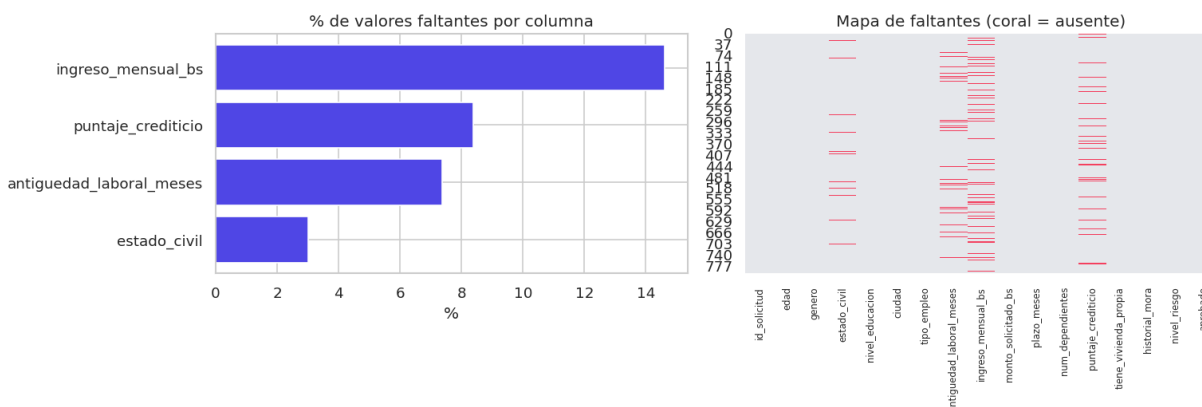


Figura. Izquierda: % de faltantes por columna. Derecha: mapa de calor (coral = celda ausente); las franjas verticales revelan si los huecos se concentran en ciertas columnas.

El mapa de calor es útil para detectar **co-ocurrencia** de faltantes (columnas que faltan juntas), lo que suele indicar un origen común del problema.

EJERCICIO • 1.2 • DIAGNOSTICAR EL MECANISMO

Para cada columna con faltantes, decide si es MCAR, MAR o estructural. **Pista:** cruza la ausencia contra `tipo_empleo`.

Solución

La técnica clave: crear una bandera booleana «¿falta?» y ver si su probabilidad *cambia* según otra variable. Si cambia mucho → MAR; si es 100 % en un grupo → estructural; si es pareja → MCAR.

```
In [7]: # ¿La ausencia de 'ingreso_mensual_bs' depende del tipo de empleo? -> MAR
tabla = df.assign(ingreso_falta=df["ingreso_mensual_bs"].isna())
print("P(ingreso faltante | tipo_empleo):")
print((tabla.groupby("tipo_empleo")["ingreso_falta"].mean()*100).round(1))
print()

# ¿'antiguedad_laboral_meses' falta SIEMPRE que está Desempleado? -> estructural
print("Faltantes de antiguedad por tipo_empleo:")
print(df.groupby("tipo_empleo")["antiguedad_laboral_meses"]
      .apply(lambda s: f"{s.isna().mean()*100:.0f}% ({s.isna().sum()})"))
```

```
Out
[6]: P(ingreso faltante | tipo_empleo):
tipo_empleo
Dependiente      7.0
Desempleado      3.4
Empleado Público  6.1
Independiente    35.3
Jubilado         3.8

Faltantes de antiguedad por tipo_empleo:
tipo_empleo
Dependiente      0% (0)
Desempleado     100% (59)
Empleado Público  0% (0)
Independiente    0% (0)
Jubilado         0% (0)
```

NOTA • LECTURA DEL RESULTADO

`ingreso_mensual_bs` falta mucho más en **Independientes** (35 % vs. 3–7 %) → **MAR** (imputar por grupo). `antiguedad_laboral_meses` falta en el **100 % de los Desempleados** → **estructural** (el valor no existe; no tiene sentido imputar una mediana). `puntaje_crediticio` falta de forma pareja → **MCAR**.

EJERCICIO • 1.3 • IMPUTAR SEGÚN EL MECANISMO

Aplica una estrategia distinta a cada caso: mediana global (MCAR), mediana por grupo (MAR) y relleno con 0 + variable indicadora (estructural). Verifica que ya no queden faltantes.

Solución

Nótese que se usa la **mediana** y no la media: la mediana es robusta ante los outliers que ya detectamos en el ingreso. Para la categórica `estado_civil` se usa la **moda** (`most_frequent`), pues la media no tiene sentido en texto.

```
In [8]: from sklearn.impute import SimpleImputer

df_imp = df.copy()

# (a) MCAR -> mediana global
imp_mediana = SimpleImputer(strategy="median")
df_imp["puntaje_crediticio"] =
imp_mediana.fit_transform(df_imp[["puntaje_crediticio"]])

# (b) MAR -> mediana CONDICIONADA al tipo de empleo
df_imp["ingreso_mensual_bs"] = (
    df_imp.groupby("tipo_empleo")["ingreso_mensual_bs"]
    .transform(lambda s: s.fillna(s.median()))
)

# (c) Estructural -> el dato no existe: marca con indicador y rellena con 0
df_imp["sin_antiguedad"] = df_imp["antiguedad_laboral_meses"].isna().astype(int)
df_imp["antiguedad_laboral_meses"] = df_imp["antiguedad_laboral_meses"].fillna(0)

# (d) Categórica MCAR -> moda
df_imp["estado_civil"] = SimpleImputer(strategy="most_frequent") \
    .fit_transform(df_imp[["estado_civil"]]).ravel()

print("Faltantes restantes:", int(df_imp.isna().sum().sum()))
```

```
Out
[7]: Faltantes restantes: 0
```

Detalles de lo aplicado: la mediana global de `puntaje_crediticio` fue 655; el ingreso se relleno con la mediana de cada `tipo_empleo`; y se creó la bandera `sin_antiguedad` (59 casos en 1) antes de poner 0 en la antigüedad.

CAUIDADO • ERROR FRECUENTE

Imputar `antiguedad_laboral_meses` con la mediana metería ~268 meses de antigüedad a personas **desempleadas**: inventarías información falsa. La variable indicadora `sin_antiguedad` conserva la señal de que el dato *no aplicaba*.

5 Codificación de variables categóricas • Unidad 2.3.2

Un modelo necesita números. Codificar es traducir etiquetas a números **sin mentirle al modelo**: sin inventar órdenes que no existen ni distancias que no significan nada.

EJERCICIO • 2.1 • LIMPIAR ANTES DE CODIFICAR

Normaliza `estado_civil` y `tiene_vivienda_propia` para eliminar diferencias de mayúsculas / espacios. ¿Cuántas categorías únicas quedan antes y después?

Solución

Las dos herramientas de limpieza de texto en pandas: `.str.strip()` (quita espacios) y `.str.title()` / `.str.lower()` (unifica mayúsculas). Para `tiene_vivienda_propia` además se usa un `.map()` para colapsar variantes a dos valores canónicos.

```
In [9]: print("Antes ->", df_imp["tiene_vivienda_propia"].nunique(), "categorías:",
        sorted(df_imp["tiene_vivienda_propia"].dropna().unique()))

df_imp["estado_civil"] = df_imp["estado_civil"].str.strip().str.title()
df_imp["tiene_vivienda_propia"] = (df_imp["tiene_vivienda_propia"]
                                   .str.strip().str.lower()
                                   .map({"sí":"Sí", "si":"Sí", "no":"No"}))

print("Después ->", df_imp["tiene_vivienda_propia"].nunique(), "categorías:",
      sorted(df_imp["tiene_vivienda_propia"].dropna().unique()))
print("estado_civil:", sorted(df_imp["estado_civil"].unique()))
```

```
Out Antes -> 4 categorías: ['NO', 'No', 'Sí', 'si']
[8]: Después -> 2 categorías: ['No', 'Sí']
      estado_civil: ['Casado', 'Divorciado', 'Soltero',
                    'Unión Libre', 'Viudo']
```

La limpieza colapsó `tiene_vivienda_propia` de 4 a 2 categorías y `estado_civil` de 10 a 5. Sin este paso, one-hot habría creado columnas duplicadas (una para «Casado» y otra para «CASADO»).

DEFINICIÓN • NOMINAL VS. ORDINAL

Nominal: categorías *sin* orden (ciudad, tipo de empleo) → `OneHotEncoder`. **Ordinal**: categorías con orden natural (nivel educativo, riesgo) → `OrdinalEncoder` con el orden **declarado explícitamente**. Codificar una nominal como ordinal le inventa un orden falso.

EJERCICIO • 2.2 • CODIFICAR VARIABLES ORDINALES

Codifica `nivel_educacion` y `nivel_riesgo` respetando su orden.

Solución

La clave de `OrdinalEncoder` es pasarle el argumento `categories=` con el orden correcto. Si se omite, scikit-learn ordena **alfabéticamente** —lo que pondría «Alto» < «Bajo» < «Medio», un sinsentido—.

```
In from sklearn.preprocessing import OrdinalEncoder
[10]: orden_educacion = ["Primaria", "Secundaria", "Técnico", "Licenciatura", "Posgrado"]
```

```
orden_riesgo = ["Bajo", "Medio", "Alto"]

ord_enc = OrdinalEncoder(categories=[orden_educacion, orden_riesgo])
df_imp[["educacion_cod", "riesgo_cod"]] = ord_enc.fit_transform(
    df_imp[["nivel_educacion", "nivel_riesgo"]])

df_imp[["nivel_educacion", "educacion_cod", "nivel_riesgo", "riesgo_cod"]].head()
```

El resultado mapea cada nivel a un entero que respeta la jerarquía (Primaria=0 ... Posgrado=4; Bajo=0, Medio=1, Alto=2):

nivel_educacion	educacion_cod	nivel_riesgo	riesgo_cod
Licenciatura	3.0	Bajo	0.0
Técnico	2.0	Alto	2.0
Posgrado	4.0	Alto	2.0
Licenciatura	3.0	Bajo	0.0
Secundaria	1.0	Bajo	0.0

EJERCICIO • 2.3 • CODIFICAR VARIABLES NOMINALES (ONE-HOT)

Aplica `OneHotEncoder` a `genero`, `estado_civil` y `tipo_empleo`. ¿Cuántas columnas nuevas se crean?

Solución

One-hot crea una columna binaria (0/1) por cada categoría: una sola de ellas vale 1 en cada fila. Dos argumentos importan: `sparse_output=False` devuelve una matriz densa legible, y `handle_unknown="ignore"` evita que el modelo falle si en producción aparece una categoría no vista en entrenamiento.

```
In from sklearn.preprocessing import OneHotEncoder
[11]:
nominales = ["genero", "estado_civil", "tipo_empleo"]
ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
mat = ohe.fit_transform(df_imp[nominales])
ohe_df = pd.DataFrame(mat, columns=ohe.get_feature_names_out(nominales),
index=df_imp.index)

print("Columnas creadas:", ohe_df.shape[1])
ohe_df.head()
```

```
Out Columnas creadas: 12
[9]: ['genero_Femenino', 'genero_Masculino',
      'estado_civil_Casado', 'estado_civil_Divorciado',
      'estado_civil_Soltero', 'estado_civil_Unión Libre',
      'estado_civil_Viudo', 'tipo_empleo_Dependiente',
      'tipo_empleo_Desempleado', 'tipo_empleo_Empleado Público',
      'tipo_empleo_Independiente', 'tipo_empleo_Jubilado']
```

Tres columnas originales (2 + 5 + 5 categorías) se convirtieron en **12 columnas** binarias. La expansión es el precio del one-hot.

EJERCICIO • 2.4 • ALTA CARDINALIDAD: ONE-HOT VS. FREQUENCY ENCODING

`ciudad` tiene 15 categorías. Compara el costo en columnas de one-hot frente a un *frequency encoding* (una sola columna con la frecuencia de cada ciudad).

Solución

El *frequency encoding* reemplaza cada categoría por su frecuencia relativa en el dataset. Comprime 15 columnas potenciales en una sola, a costa de perder identidad (dos ciudades igual de frecuentes reciben el mismo número).

```
In # One-hot de ciudad -> muchas columnas
[12]: n_oh_ciu = df_imp["ciudad"].nunique()

# Frequency encoding -> una sola columna
frec = df_imp["ciudad"].value_counts(normalize=True)
df_imp["ciudad_frec"] = df_imp["ciudad"].map(frec)

print(f"One-hot de 'ciudad' -> {n_oh_ciu} columnas nuevas")
print("Frequency encoding -> 1 columna")
df_imp[["ciudad", "ciudad_frec"]].drop_duplicates().sort_values("ciudad_frec",
ascending=False).head()
```

Comparación de costo y frecuencias por ciudad (top 5):

ciudad	ciudad_frec
La Paz	0.196
Santa Cruz	0.188
Cochabamba	0.149
El Alto	0.135
Potosí	0.076

`ciudad` en one-hot = **15 columnas**; en frequency encoding = **1 columna**.

NOTA • CRITERIO

Con **pocas** categorías, one-hot es lo más seguro. Con **alta cardinalidad** (decenas o cientos), one-hot explota el número de columnas; ahí conviene *frequency / target encoding* o agrupar categorías raras en «Otros».

6 Escalado y normalización · Unidad 2.3.3

DEFINICIÓN · POR QUÉ ESCALAR

Modelos basados en distancias o gradientes (KNN, SVM, regresión logística, redes) se dejan **dominar** por las variables de mayor magnitud. `ingreso_mensual_bs` (hasta ~185.000) aplastaría a `edad` (18–70) si no se escalan a un rango comparable.

Las tres fórmulas

Los tres escaladores más usados y su fórmula:

StandardScaler (z-score): $z = (x - \mu) / \sigma$

centra en media 0 y desviación 1

MinMaxScaler: $x' = (x - \text{mín}) / (\text{máx} - \text{mín})$

comprime todo al rango [0, 1]

RobustScaler: $x' = (x - \text{mediana}) / \text{IQR}$

IQR = Q3 – Q1 · usa estadísticos resistentes a outliers

EJERCICIO · 3.1 · COMPARAR LOS TRES ESCALADORES

Sobre `ingreso_mensual_bs` (que tiene *outliers*), compara `StandardScaler`, `MinMaxScaler` y `RobustScaler`. ¿Cuál resiste mejor los valores extremos?

Solución

```
In [13]: from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

x = df_imp[["ingreso_mensual_bs"]].values
escaladores = {"StandardScaler":StandardScaler(),
               "MinMaxScaler":MinMaxScaler(),
               "RobustScaler":RobustScaler()}

fig, axes = plt.subplots(1, 4, figsize=(15, 3.5))
axes[0].hist(x, bins=40, color="#9CA3AF"); axes[0].set_title("Original")
for ax,(nombre,esc),col in zip(axes[1:], escaladores.items()),
[INDIGO,PURPURA,CORAL]):
    ax.hist(esc.fit_transform(x), bins=40, color=col); ax.set_title(nombre)
plt.tight_layout(); plt.show()

resumen = pd.DataFrame({n: esc.fit_transform(x).ravel() for n,esc in
escaladores.items()})
resumen.describe().round(2)
```

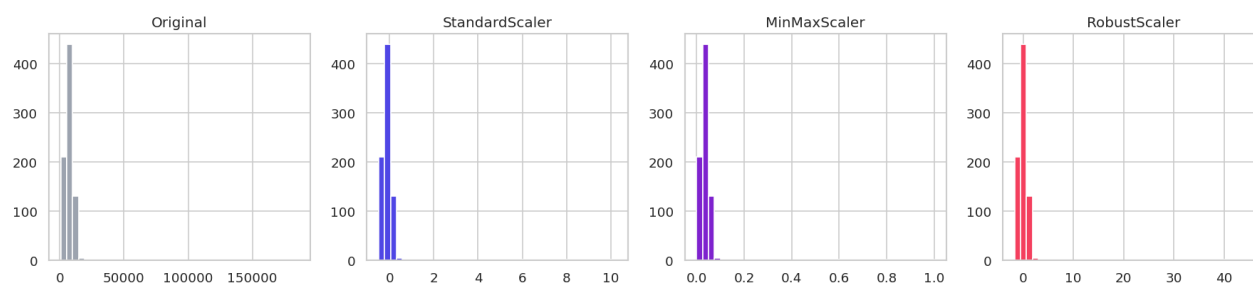


Figura. La forma de la distribución no cambia; cambia la escala del eje. El objetivo es comparar cómo cada escalador ubica los mismos datos.

El resumen estadístico de la variable tras cada escalado:

estadístico	StandardScaler	MinMaxScaler	RobustScaler
media	0.00	0.05	0.48
std	1.00	0.09	4.40
mín	-0.50	0.00	-1.70
25%	-0.24	0.02	-0.55
50% (mediana)	-0.11	0.04	0.00
75%	-0.01	0.05	0.45
máx	10.26	1.00	45.62

NOTA • LECTURA

Los *outliers* comprimen a **MinMaxScaler** (casi todo cae entre 0.02 y 0.05, aplastado contra el 0) y estiran la desviación de **StandardScaler** (el máximo llega a 10 σ). **RobustScaler** usa mediana e IQR → deja la mediana en 0 y es mucho menos sensible a los extremos. **Con outliers, RobustScaler es la opción más segura.**

EJERCICIO • 3.2 • ESCALAR SIN FUGA DE DATOS

Divide en train / test y ajusta (**fit**) el escalador **solo con train**. Explica por qué usar todo el dataset sería un error.

Solución

Regla de oro: **fit** (aprender los parámetros: media, desviación, mín, máx...) **solo con train**; **transform** (aplicarlos) sobre train y test. El test debe simular datos nunca vistos, así que no puede participar en el cálculo de los parámetros.

```
In from sklearn.model_selection import train_test_split
[14]:
    num_cols = ["edad", "ingreso_mensual_bs", "monto_solicitado_bs", "puntaje_credicio",
                "antiguedad_laboral_meses"]
    X_tr, X_te = train_test_split(df_imp[num_cols], test_size=0.25,
                                random_state=42, stratify=df_imp["aprobado"])

    sc = StandardScaler().fit(X_tr)          # <-- fit SOLO con train
    X_tr_s = sc.transform(X_tr)
    X_te_s = sc.transform(X_te)             # test se transforma con parámetros de
    train
```

```
print("Media aprendida (train):", sc.mean_.round(1))
print("Media del test tras escalar (no es exactamente 0, y está bien):",
      X_te_s.mean(axis=0).round(3))
```

```
Out      Media aprendida (train): [  44.6  8775.5 56650.9  655.1  236.1]
[11]:    Media del test tras escalar (no es 0 exacto, y está bien):
        [-0.052  0.161  0.088 -0.094 -0.006]
```

Que la media del test escalado *no* sea exactamente 0 es precisamente la **prueba** de que se hizo bien: el test se transformó con parámetros del train, no de sí mismo.

CUIDADO · DATA LEAKAGE

Si haces `fit` con **todo** el dataset, la media / desviación del **test** se filtra al entrenamiento y las métricas salen **optimistas**. El modelo parecerá mejor en el laboratorio de lo que será en producción. El test debe simular datos *nunca vistos*.

7 Pipelines de transformación • Unidad 2.3.4

DEFINICIÓN • PIPELINE + COLUMNTRANSFORMER

Un **Pipeline** encadena pasos (imputar → escalar → modelar) en un **solo objeto**. Un **ColumnTransformer** aplica transformaciones *distintas* a distintos grupos de columnas. Juntos, todo el preprocesamiento se ajusta solo con `train` y se reaplica sin fuga.

Hasta aquí hicimos cada paso a mano. Eso es frágil: es fácil olvidar reaplicar una transformación al test, o hacerla en distinto orden. El **Pipeline** resuelve esto empaquetando **todo** el flujo en un objeto que se comporta como un modelo más: tiene `.fit()` y `.predict()`, y por dentro respeta el orden y evita la fuga.

Arquitectura de tres ramas

El **ColumnTransformer** enruta cada grupo de columnas a su propia sub-pipeline:

- **Rama numérica:** imputar (mediana) → escalar (**StandardScaler**).
- **Rama nominal:** imputar (moda) → **OneHotEncoder**.
- **Rama ordinal:** imputar (moda) → **OrdinalEncoder** con orden explícito.

EJERCICIO • 4.1 • CONSTRUIR EL PIPELINE COMPLETO

Partiendo del CSV **crudo** (df original, con faltantes y texto sucio), arma un **ColumnTransformer** con tres ramas (numérica, nominal, ordinal) dentro de un **Pipeline** que termine en un clasificador, y evalúalo.

Solución — partimos de nuevo desde el CSV crudo para que el pipeline haga todo

Nota importante: la limpieza de *texto* (unificar mayúsculas) se hace **antes**, porque el **ColumnTransformer** no incluye un paso de normalización de cadenas. Todo lo demás —imputar, codificar, escalar— vive dentro del pipeline.

```
In from sklearn.pipeline import Pipeline
[15]: from sklearn.compose import ColumnTransformer
      from sklearn.linear_model import LogisticRegression
      from sklearn.metrics import classification_report, confusion_matrix

      # --- Datos crudos ---
      data = pd.read_csv("solicitudes_credito.csv")

      # Limpieza mínima de texto sucio (esto NO lo hace el ColumnTransformer)
      data["estado_civil"] = data["estado_civil"].str.strip().str.title()
      data["tiene_vivienda_propia"] = (data["tiene_vivienda_propia"].str.strip().str.lower()
                                      .map({"sí": "Sí", "si": "Sí", "no": "No"}))

      y = data["aprobado"]
      X = data.drop(columns=["aprobado", "id_solicitud"])

      num_cols = ["edad", "ingreso_mensual_bs", "monto_solicitado_bs", "puntaje_crediticio",
                  "antigüedad_laboral_meses", "num_dependientes", "plazo_meses"]
```

```

nom_cols =
["genero", "estado_civil", "tipo_empleo", "ciudad", "tiene_vivienda_propia", "historial_mora"]
ord_cols = ["nivel_educacion", "nivel_riesgo"]

In # --- Tres ramas de preprocesamiento ---
[16]: rama_num = Pipeline([
      ("imputar", SimpleImputer(strategy="median")),
      ("escalar", StandardScaler()),
    ])
    rama_nom = Pipeline([
      ("imputar", SimpleImputer(strategy="most_frequent")),
      ("onehot", OneHotEncoder(sparse_output=False, handle_unknown="ignore")),
    ])
    rama_ord = Pipeline([
      ("imputar", SimpleImputer(strategy="most_frequent")),
      ("ordinal", OrdinalEncoder(categories=[orden_educacion, orden_riesgo],
                                     handle_unknown="use_encoded_value", unknown_value=-1)),
    ])

    preprocesador = ColumnTransformer([
      ("num", rama_num, num_cols),
      ("nom", rama_nom, nom_cols),
      ("ord", rama_ord, ord_cols),
    ])

    modelo = Pipeline([
      ("preprocesamiento", preprocesador),
      ("clasificador", LogisticRegression(max_iter=1000)),
    ])
    modelo

```

El objeto `modelo` resultante es un pipeline anidado: preprocesamiento (con sus tres ramas) seguido de un `LogisticRegression`. Ahora se entrena y evalúa como cualquier modelo:

```

In # --- Entrenar y evaluar ---
[17]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.25, random_state=42, stratify=y)

      modelo.fit(X_train, y_train)
      y_pred = modelo.predict(X_test)

      print(classification_report(y_test, y_pred, target_names=["Rechazado", "Aprobado"]))

```

Reporte de clasificación sobre el conjunto de prueba (200 solicitudes):

clase	precision	recall	f1-score	support
Rechazado	0.68	0.65	0.66	118
Aprobado	0.52	0.55	0.54	82
accuracy			0.61	200
macro avg	0.60	0.60	0.60	200
weighted avg	0.61	0.61	0.61	200

```

In # Matriz de confusión
[18]: cm = confusion_matrix(y_test, y_pred)
      fig, ax = plt.subplots(figsize=(4.2, 3.6))
      sns.heatmap(cm, annot=True, fmt="d", cmap="Purples", cbar=False,

```

clase	precision	recall	f1-score	support
<pre> xticklabels=["Rechazado", "Aprobado"], yticklabels=["Rechazado", "Aprobado"], ax=ax) ax.set_xlabel("Predicho"); ax.set_ylabel("Real"); ax.set_title("Matriz de confusión") plt.tight_layout(); plt.show() </pre>				

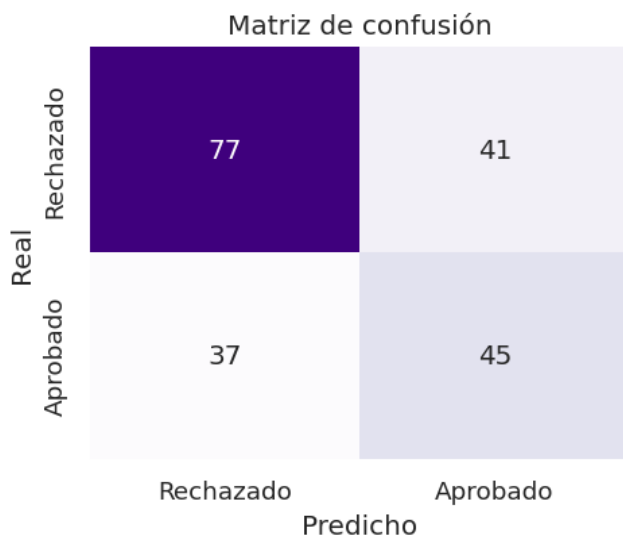


Figura. Matriz de confusión: la diagonal son los aciertos. El modelo es honesto (~61 % de acierto); lo importante de esta guía no es la métrica, sino que el preprocesamiento se hizo sin fuga.

NOTA • LO IMPORTANTE DEL PIPELINE

El objeto `modelo` guarda dentro de sí **todo**: imputación, escalado y codificación se ajustaron *solo con train* y se reaplican automáticamente al test y a cualquier dato futuro. Cero fuga, cero pasos manuales que olvidar.

EJERCICIO • 4.2 • VALIDACIÓN CRUZADA

Estima el desempeño con validación cruzada de 5 particiones sobre el pipeline completo. ¿Por qué es seguro hacer CV con un pipeline y peligroso hacerlo con datos ya preprocesados a mano?

Solución

```

In [19]: from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y, cv=5, scoring="f1")
print("F1 por partición:", scores.round(3))
print(f"F1 promedio: {scores.mean():.3f} ± {scores.std():.3f}")

```

```

Out [15]: F1 por partición: [0.641 0.552 0.585 0.558 0.647]
F1 promedio: 0.597 ± 0.040

```

Un F1 promedio de 0.597 ± 0.040 es una estimación mucho más confiable que un único train/test, porque promedia sobre cinco divisiones distintas y reporta la variabilidad.

CUIDADO • POR QUÉ EL PIPELINE ES CLAVE EN CV

En cada partición de CV, el preprocesamiento se **reajusta** usando **solo** el train de esa partición. Si escalas / imputas *antes* de la CV, cada fold ya vio información del resto → fuga sutil que infla las métricas.

8 Síntesis

El mapa completo de decisiones de esta guía:

Subtema	Decisión clave	Herramienta
2.3.1 Faltantes	La estrategia depende del mecanismo (MCAR / MAR / estructural).	<code>SimpleImputer</code> , imputación por grupo, indicador
2.3.2 Codificación	Nominal → one-hot; ordinal → orden explícito; alta cardinalidad → frequency.	<code>OneHotEncoder</code> , <code>OrdinalEncoder</code>
2.3.3 Escalado	Con <i>outliers</i> → <code>RobustScaler</code> ; siempre <code>fit</code> solo con train.	<code>Standard/MinMax/RobustScaler</code>
2.3.4 Pipelines	Todo el preprocesamiento vive dentro del <code>Pipeline</code> → sin fuga.	<code>Pipeline</code> , <code>ColumnTransformer</code>

Reglas de oro para llevarse

- **Radiografía primero:** `info()` + `describe()` antes de tocar nada.
- **Pregúntate por qué falta el dato**, no solo con qué rellenarlo.
- **Limpia el texto antes de codificar**; nunca dejes que «CASADO» y «Casado» sean categorías distintas.
- **Nunca conviertas una nominal en ordinal** sin un orden real.
- **Con outliers, escala robusto**; y siempre `fit` solo con train.
- **Empaqueta todo en un Pipeline**: es la única defensa fiable contra la fuga de datos.

9 Glosario de términos

Término	Significado
MCAR	Missing Completely At Random — la ausencia no depende de ninguna variable.
MAR	Missing At Random — la ausencia depende de otra variable observada.
MNAR / estructural	La ausencia depende del propio valor, o el dato no existe para esa fila.
Imputar	Rellenar valores faltantes con un valor estimado (media, mediana, moda, por grupo...).
One-hot encoding	Una columna binaria por categoría; solo una vale 1 por fila.
Ordinal encoding	Asignar enteros que respetan un orden natural declarado.
Frequency encoding	Reemplazar cada categoría por su frecuencia relativa.
Cardinalidad	Número de categorías distintas de una variable.
IQR	Rango intercuartílico, $Q3 - Q1$; base del <code>RobustScaler</code> .
Outlier	Valor extremo, muy alejado del grueso de la distribución.
Data leakage	Fuga: información del test (o del futuro) se filtra al entrenamiento e infla las métricas.

Término	Significado
Pipeline	Objeto que encadena pasos de preprocesamiento y modelo en una sola unidad reproducible.
ColumnTransformer	Aplica transformaciones distintas a distintos grupos de columnas.
Validación cruzada (CV)	Evaluar el modelo en varias particiones train/test y promediar el resultado.

10 Referencias

- Documentación oficial de scikit-learn — [sklearn.preprocessing](#), [sklearn.impute](#), [sklearn.compose.ColumnTransformer](#), [sklearn.pipeline.Pipeline](#).
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* — cap. 2 (End-to-End ML Project), sección de preparación de datos.
- McKinney, W. *Python for Data Analysis* — manejo de datos faltantes y transformación de columnas en pandas.
- Little, R. J. A. & Rubin, D. B. *Statistical Analysis with Missing Data* — marco formal MCAR / MAR / MNAR.

— Fin de la GL07 • INF672 Machine Learning —